

Exam 19-06-2026 - Selected Solutions

Problem 1: Project Selection (Max-Flow Min-Cut)

1. Max-Flow Min-Cut Model

We construct a directed graph $G = (V, E)$ to model the projects and their dependencies: * **Nodes:** $V = 1, \dots, n \cup s, t$, where s is the source and t is the sink. * **Profitable Projects:** For each project j with $p_j > 0$, add an edge (s, j) with capacity $c(s, j) = p_j$. * **Costly Projects / Enablers:** For each project j with $p_j \leq 0$, add an edge (j, t) with capacity $c(j, t) = -p_j$. * **Dependencies:** If project j requires project i (i.e., $(j, i) \in D$), add an edge (j, i) with infinite capacity $c(j, i) = \infty$. This prevents selecting j without i , as cutting this edge would yield an infinite capacity cut.

2. Algorithm

1. Compute the Maximum Flow from s to t using a standard algorithm (e.g., Edmonds-Karp or Dinic).
2. Find the Minimum Cut (A, B) associated with this flow, where $s \in A$ and $t \in B$.
3. The optimal feasible selection of projects is $S = A \setminus s$. This set can be found by performing a BFS or DFS from s in the residual graph to find all reachable nodes.

3. Computational Complexity

If we use the **Edmonds-Karp** algorithm, the time complexity to find the max flow is $O(|V| \cdot |E|^2)$. In our graph, $|V| = n + 2 = O(n)$ and $|E| \leq n + |D| = O(n + |D|)$. Therefore, the final complexity is $O(n \cdot (n + |D|)^2)$.

Problem 2: Maximum Contiguous Subarray

1. Recursive Formula

Let $OPT[i]$ be the maximum value of a contiguous subarray that *ends exactly* at position i . At position i , we can either append $A[i]$ to the optimal subarray ending at $i - 1$, or start a completely new subarray containing only $A[i]$. To maximize the sum, we take the maximum of these two choices:

$$OPT[i] = \max(A[i], OPT[i - 1] + A[i])$$

Base Case: $OPT[1] = A[1]$.

2. Efficient Implementation and Complexity

We only need the previous value $OPT[i - 1]$ to compute $OPT[i]$, so we can optimize space by using a single variable `curr_max`. We track the global maximum with `max_so_far`.

```

Algorithm MaxSubarray(A, n):
    curr_max = A[1]
    max_so_far = A[1]
    For i = 2 to n:
        curr_max = max(A[i], curr_max + A[i])
        if curr_max > max_so_far:
            max_so_far = curr_max
    Return max_so_far

```

- **Time Complexity:** $O(n)$ because we iterate through the array of length n exactly once, performing $O(1)$ operations per step.
- **Space Complexity:** $O(1)$ since we only store a few scalar variables, regardless of the array size.

3. Computing the Actual Subarray

To find the actual indices, we maintain a temporary start index that updates whenever we decide to start a new subarray. We update the global indices only when a new `max_so_far` is found.

```

Algorithm MaxSubarrayIndices(A, n):
    curr_max = max_so_far = A[1]
    temp_start = global_start = global_end = 1
    For i = 2 to n:
        if A[i] > curr_max + A[i]:
            curr_max = A[i]
            temp_start = i
        else:
            curr_max = curr_max + A[i]

        if curr_max > max_so_far:
            max_so_far = curr_max
            global_start = temp_start
            global_end = i
    Return (max_so_far, global_start, global_end)

```

Problem 3: k-Center Problem

1. Definition and Poor Performance of $k=1$

Definition: Given a set of points V in a metric space (with a distance function d satisfying the triangle inequality) and an integer k , the k -center problem asks to find a set $C \subseteq V$ of k centers to minimize the maximum distance from any point in V to its nearest center. Objective:

$$\min_{C, |C|=k} \max_{v \in V} d(v, C).$$

Example: Consider a metric space with 3 locations A, B, C . Let $d(A, B) = 10$, $d(B, C) = 10$, and $d(A, C) = 20$. We place 100 overlapping points at location A , 1 point at B , and 100 overlapping points at C . * **Optimal $k = 1$:** Placing the center at B gives a maximum radius of 10. (Placing it at A or C would give a radius of 20). * **Optimal $k = 2$:** Placing centers at A, C gives a maximum radius of 10 (for the single point B). * **Poor Performance:** If we force the optimal $k = 1$ center (B) to be part of the $k = 2$ solution, our centers are B, A or B, C . In either case, the remaining uncentered cluster (C or A) is at distance 20 from its nearest center. The radius is 20, which is 2 times worse than the optimal $k = 2$ solution (radius 10).

2. NP-Hardness

We can prove k -center is NP-hard by a reduction from the **Dominating Set** problem. Given an instance of Dominating Set $\langle G = (V, E), k \rangle$, we construct a metric space on V . We define the distance $d(u, v) = 1$ if $(u, v) \in E$, and $d(u, v) = 2$ if $(u, v) \notin E$ (with $d(u, u) = 0$). This satisfies the triangle inequality. * If G has a dominating set of size k , selecting those k vertices as centers yields a max radius of 1. * If G does not have a dominating set of size k , any choice of k centers leaves at least one node non-adjacent to all centers, resulting in a max radius of 2. Thus, finding the optimal k -center radius solves the NP-complete Dominating Set problem.

3. 2-Approximation Algorithm (Gonzalez's Algorithm)

Algorithm: 1. Choose an arbitrary point $c_1 \in V$ and add it to the set of centers C . 2. For $i = 2$ to k : * Find the point $v \in V$ that is farthest from the current set of centers (i.e., maximizes $d(v, C)$). * Add v to C . 3. Return C .

Proof of 2-Approximation: Let r be the maximum distance from any point to C at the end of the algorithm. This means there exists some point $v_{k+1} \in V$ such that its distance to all k centers in C is exactly r . Consider the set of $k + 1$ points $S = C \cup v_{k+1}$. By construction, the distance between *any* pair of points in S is at least r . The optimal solution OPT uses exactly k centers to cover all points. By the Pigeonhole Principle, at least two points from our set S (which has $k + 1$ points) must be assigned to the same optimal center. Let these two points be $x, y \in S$. Their distance to this optimal center is at most OPT . By the triangle inequality, $d(x, y) \leq d(x, center) + d(center, y) \leq 2 \cdot OPT$. Since we know the distance between *any* two points in S is at least r , we have $r \leq d(x, y) \leq 2 \cdot OPT$. Thus, the algorithm's radius r is at most twice the optimal radius.

Problem 4: Max3SAT

1. Randomized Approximation Algorithm

Algorithm: Assign each variable $x_i \in X$ to **True** or **False** independently and uniformly at random (with probability $1/2$). **Time Complexity:** The algorithm requires $O(n)$ time to assign a truth value to all n variables, and $O(k)$ time to evaluate the k clauses. The total time complexity is $O(n + k)$.

2. Expected Approximation Ratio

Let C_j be a clause in the 3-CNF formula. Since it is a 3SAT problem, C_j contains exactly 3 literals. The clause C_j evaluates to **False** if and only if *all three* of its literals evaluate to **False**. Since the variables are assigned independently, the probability that a specific literal is false is $1/2$.

$$\Pr[C_j \text{ is False}] = \left(\frac{1}{2}\right)^3 = \frac{1}{8}$$

$$\Pr[C_j \text{ is Satisfied}] = 1 - \frac{1}{8} = \frac{7}{8}$$

Let Z_j be an indicator random variable that is 1 if C_j is satisfied and 0 otherwise. The total number of satisfied clauses is $Z = \sum_{j=1}^k Z_j$. By the linearity of expectation:

$$\mathbb{E}[Z] = \sum_{j=1}^k \mathbb{E}[Z_j] = \sum_{j=1}^k \frac{7}{8} = \frac{7}{8}k$$

Since the optimal solution OPT can satisfy at most all k clauses ($OPT \leq k$), the expected approximation ratio is:

$$\frac{\mathbb{E}[Z]}{OPT} \geq \frac{\frac{7}{8}k}{k} = \frac{7}{8}$$

3. Integer Linear Programming (ILP) Formulation

- **Variables:**
- Let $y_i \in \{0, 1\}$ for each variable x_i (1 if True, 0 if False).
- Let $z_j \in \{0, 1\}$ for each clause C_j (1 if Satisfied, 0 if not).
- **Objective Function:** Maximize the number of satisfied clauses:

$$\max \sum_{j=1}^k z_j$$

- **Constraints:** For each clause $C_j = l_{j,1} \vee l_{j,2} \vee l_{j,3}$, we map each literal to its corresponding variable expression. Let $v(l)$ be defined as:
 - $v(l) = y_i$ se il letterale l è la variabile positiva x_i .
 - $v(l) = 1 - y_i$ se il letterale l è la variabile negata $\neg x_i$.

We add the following constraint for each clause j :

$$z_j \leq v(l_{j,1}) + v(l_{j,2}) + v(l_{j,3})$$

This ensures that z_j can only be 1 (satisfied) if at least one of the literals in the clause evaluates to 1. If all literals evaluate to 0, the right side is 0, forcing $z_j = 0$.

$$y_i \in \{0, 1\} \quad \forall i = 1, \dots, n$$

$$z_j \in \{0, 1\} \quad \forall j = 1, \dots, k$$